

PCA - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Step 1: Importing Libraries

Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import ipywidgets as widgets
from IPython.display import display
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from ipywidgets import interact, IntSlider
print("Libraries Imported")
```

Output:

Libraries Imported

Step 2: Loading Dataset

Load the Breast cancer Wisconsin dataset

```
df = pd.read_csv('Breast_cancer_Wisconsin_data.csv')
print("Dataset loaded successfully")
```

Output:

Dataset loaded successfully

Step 3: Data Analysis

Display the first 5 rows of the dataset

```
df.head()
```

Output:

```

id      radius_mean texture_mean perimeter_mean area_mean smoothness_mean
compactness_mean concavity_mean concave points_mea symmetry_mean ... texture_worst
perimeter_worst area_worst smoothness_worst compactness_worst concavity_worst concave
points_wor symmetry_worst fractal_dimension_ diagnosis
0  842302    17.99      10.38      122.80      1001.0      0.11840      0.27760
0.3001      0.14710      0.2419      ... 17.33      184.60      2019.0
0.1622      0.6656      0.7119      0.2654      0.4601
0.11890      M
1  842517    20.57      17.77      132.90      1326.0      0.08474      0.07864
0.0869      0.07017      0.1812      ... 23.41      158.80      1956.0
0.1238      0.1866      0.2416      0.1860      0.2750
0.08902      M
2  84300903  19.69      21.25      130.00      1203.0      0.10960      0.15990
0.1974      0.12790      0.2069      ... 25.53      152.50      1709.0
0.1444      0.4245      0.4504      0.2430      0.3613
0.08758      M
3  84348301  11.42      20.38      77.58      386.1      0.14250      0.28390
0.2414      0.10520      0.2597      ... 26.50      98.87      567.7
0.2098      0.8663      0.6869      0.2575      0.6638
0.17300      M
4  84358402  20.29      14.34      135.10      1297.0      0.10030      0.13280
0.1980      0.10430      0.1809      ... 16.67      152.20      1575.0
0.1374      0.2050      0.4000      0.1625      0.2364
0.07678      M
5 rows x 32 columns

```

Displays the last five rows of the dataset

```
df.tail()
```

Output:

```

id      radius_mean texture_mean perimeter_mean area_mean smoothness_mean compactness_mean
concavity_mean concave points_mea symmetry_mean ... texture_worst perimeter_worst
area_worst smoothness_worst compactness_worst concavity_worst concave points_wor
symmetry_worst fractal_dimension_ diagnosis
564  926424  21.56      22.39      142.00      1479.0      0.11100      0.11590
0.24390      0.13890      0.1726      ... 26.40      166.10      2027.0
0.14100      0.21130      0.4107      0.2216      0.2060
0.07115      M
565  926682  20.13      28.25      131.20      1261.0      0.09780      0.10340
0.14400      0.09791      0.1752      ... 38.25      155.00      1731.0
0.11660      0.19220      0.3215      0.1628      0.2572
0.06637      M
566  926954  16.60      28.08      108.30      858.1      0.08455      0.10230
0.09251      0.05302      0.1590      ... 34.12      126.70      1124.0
0.11390      0.30940      0.3403      0.1418      0.2218
0.07820      M
567  927241  20.60      29.33      140.10      1265.0      0.11780      0.27700
0.35140      0.15200      0.2397      ... 39.42      184.60      1821.0
0.16500      0.86810      0.9387      0.2650      0.4087
0.12400      M
568  92751   7.76      24.54      47.92      181.0      0.05263      0.04362
0.00000      0.00000      0.1587      ... 30.37      59.16      268.6
0.08996      0.06444      0.0000      0.0000      0.2871
0.07039      B
5 rows x 32 columns

```

Check dataset shape

```
df.shape
```

Output:
(569, 32)

Displays summary of dataset, including column names, data types, and non-null counts.

```
df.info()
```

Output:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):

#	Column	Non-Null Count	Dtype
0	id	569 non-null	int64
1	radius_mean	569 non-null	float64
2	texture_mean	569 non-null	float64
3	perimeter_mean	569 non-null	float64
4	area_mean	569 non-null	float64
5	smoothness_mean	569 non-null	float64
6	compactness_mean	569 non-null	float64
7	concavity_mean	569 non-null	float64
8	concave points_mean	569 non-null	float64
9	symmetry_mean	569 non-null	float64
10	fractal_dimension_	569 non-null	float64
11	radius_se	569 non-null	float64
12	texture_se	569 non-null	float64
13	perimeter_se	569 non-null	float64
14	area_se	569 non-null	float64
15	smoothness_se	569 non-null	float64
16	compactness_se	569 non-null	float64
17	concavity_se	569 non-null	float64
18	concave points_se	569 non-null	float64
19	symmetry_se	569 non-null	float64
20	fractal_dimension_	569 non-null	float64
21	radius_worst	569 non-null	float64
22	texture_worst	569 non-null	float64
23	perimeter_worst	569 non-null	float64
24	area_worst	569 non-null	float64
25	smoothness_worst	569 non-null	float64
26	compactness_worst	569 non-null	float64
27	concavity_worst	569 non-null	float64
28	concave points_worst	569 non-null	float64
29	symmetry_worst	569 non-null	float64
30	fractal_dimension_	569 non-null	float64
31	diagnosis	569 non-null	object

dtypes: float64(30), int64(1), object(1)
memory usage: 142.4+ KB

Statistical summary

```
df.describe()
```

Output:

```

id      radius_mean  texture_mean  perimeter_mean  area_mean  smoothness_mean
compactness_mean  concavity_mean  concave points_mea  symmetry_mean  ...  radius_worst
texture_worst  perimeter_worst  area_worst  smoothness_worst  compactness_worst
concavity_worst  concave points_wor  symmetry_worst  fractal_dimension_
count  5.690000e+02  569.000000  569.000000  569.000000  569.000000  569.000000
569.000000  569.000000  569.000000  569.000000  ...  569.000000
569.000000  569.000000  569.000000  569.000000  569.000000  569.000000
569.000000  569.000000  569.000000
mean    3.037183e+07  14.127292  19.289649  91.969033  654.889104  0.096360
0.104341  0.088799  0.048919  0.181162  ...  16.269190
25.677223  107.261213  880.583128  0.132369  0.254265  0.272188
0.114606  0.290076  0.083946
std     1.250206e+08  3.524049  4.301036  24.298981  351.914129  0.014064
0.052813  0.079720  0.038803  0.027414  ...  4.833242
6.146258  33.602542  569.356993  0.022832  0.157336  0.208624
0.065732  0.061867  0.018061
min     8.670000e+03  6.981000  9.710000  43.790000  143.500000  0.052630
0.019380  0.000000  0.000000  0.106000  ...  7.930000
12.020000  50.410000  185.200000  0.071170  0.027290  0.000000
0.000000  0.156500  0.055040
25%     8.692180e+05  11.700000  16.170000  75.170000  420.300000  0.086370
0.064920  0.029560  0.020310  0.161900  ...  13.010000
21.080000  84.110000  515.300000  0.116600  0.147200  0.114500
0.064930  0.250400  0.071460
50%     9.060240e+05  13.370000  18.840000  86.240000  551.100000  0.095870
0.092630  0.061540  0.033500  0.179200  ...  14.970000
25.410000  97.660000  686.500000  0.131300  0.211900  0.226700
0.099930  0.282200  0.080040
75%     8.813129e+06  15.780000  21.800000  104.100000  782.700000  0.105300
0.130400  0.130700  0.074000  0.195700  ...  18.790000
29.720000  125.400000  1084.000000  0.146000  0.339100  0.382900
0.161400  0.317900  0.092080
max     9.113205e+08  28.110000  39.280000  188.500000  2501.000000  0.163400
0.345400  0.426800  0.201200  0.304000  ...  36.040000
49.540000  251.200000  4254.000000  0.222600  1.058000  1.252000
0.291000  0.663800  0.207500
8 rows x 31 columns

```

Counts the frequency of each unique category

```
df['diagnosis'].value_counts()
```

Output:

```

count
diagnosis
B      357
M      212
Name: count, dtype: int64

```

Step 4: Data Preprocessing

Separate features and target variable

```
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']
print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

Output:

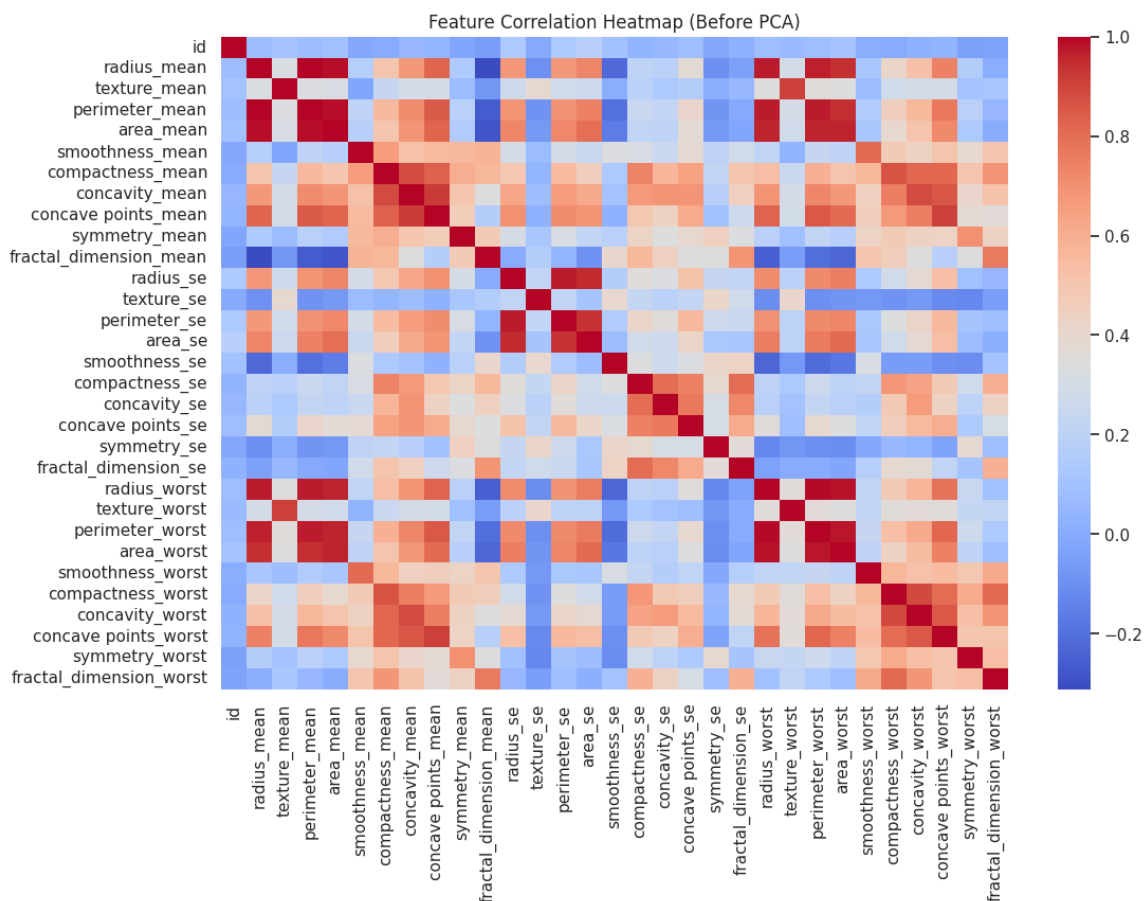
```
Feature shape: (569, 31)
Target shape: (569,)
```

Visualize correlations among features before applying PCA

```
plt.figure(figsize=(12,8))
sns.heatmap(X.corr(), cmap='coolwarm')
plt.title("Feature Correlation Heatmap (Before PCA)")
plt.show()
```

Output:

```
Feature Correlation Heatmap (Before PCA)
```

**# Standardize the features**

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Features standardized using StandardScaler")
```

Output:

```
Features standardized using StandardScaler
```

Step 5: Model Training

Data Splitting

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
print("Data has been Splitted")
```

Output:

Data has been Splitted

Apply PCA, and visualize variance

```
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
print("PCA applied with 95% variance retained.")
```

Output:

PCA applied with 95% variance retained.

Model training using Logistic Regression

```
clf = LogisticRegression(max_iter=2000)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
acc_before = accuracy_score(y_test, y_pred)
clf_pca = LogisticRegression(max_iter=2000)
clf_pca.fit(X_train_pca, y_train)
y_pred_pca = clf_pca.predict(X_test_pca)
acc_after = accuracy_score(y_test, y_pred_pca)
print("Model Training completed")
```

Output:

Model Training completed

Step 6: Model Evaluation

Compare classification accuracy before and after applying PCA

```

print("Accuracy BEFORE PCA:", round(acc_before*100, 2), "%")
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)
print("Reduced Dimensions:", X_pca.shape[1])
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.2, random_state=42, stratify=y
)
print("Accuracy AFTER PCA:", round(acc_after*100, 2), "%")
plt.figure(figsize=(6,4))
sns.barplot(
    x=['Before PCA', 'After PCA'],
    y=[acc_before, acc_after]
)
plt.ylabel("Accuracy")
plt.title("Classification Accuracy Comparison")
plt.ylim(0.9, 1.0)
plt.show()

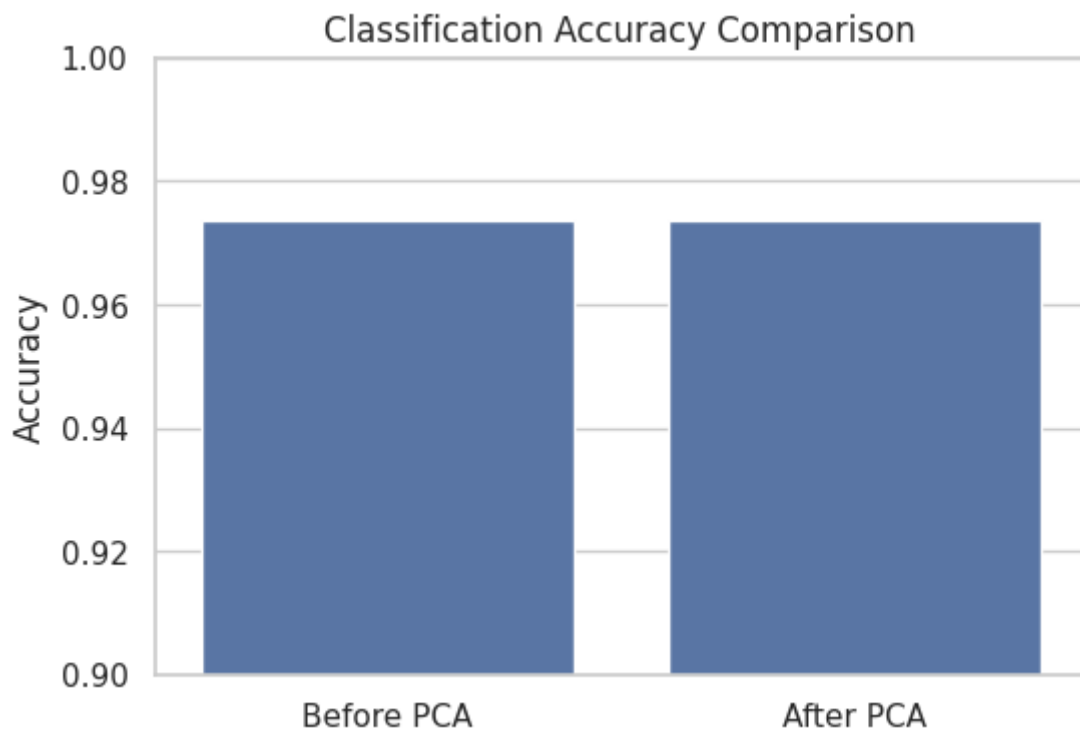
```

Output:

```

Accuracy BEFORE PCA: 97.37 %
Reduced Dimensions: 11
Accuracy AFTER PCA: 97.37 %

```

**# Apply PCA and compute feature loadings for each principal component**

```

pca = PCA(n_components=10)
X_pca = pca.fit_transform(X_scaled)
loadings = pd.DataFrame( pca.components_.T, columns=[f"PC{i+1}" for i in range(10)],
    index=X.columns)
print("PCA feature loadings calculated for the top 10 principal components.")

```

Output:

```

PCA feature loadings calculated for the top 10 principal components.

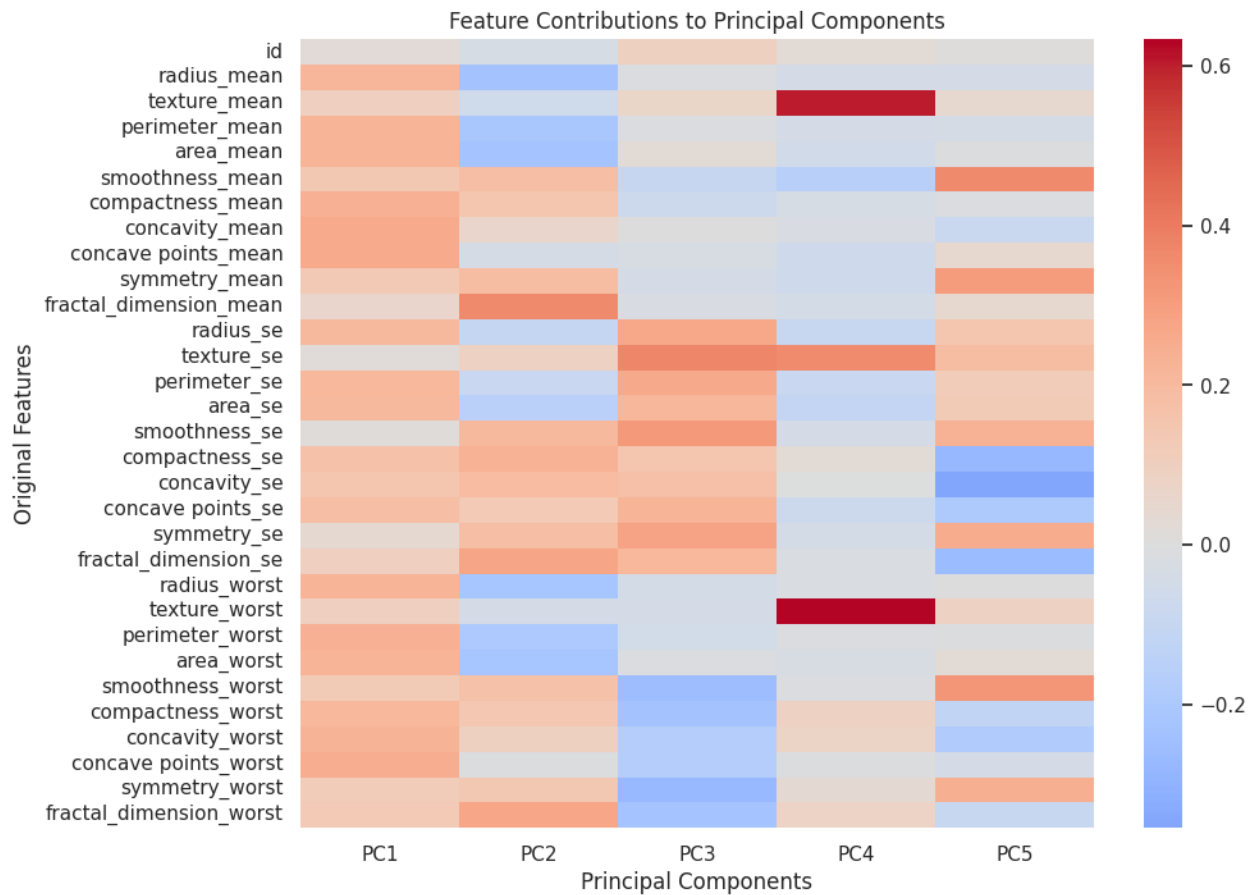
```

Visualize feature contributions to the first few principal components using a heatmap

```
plt.figure(figsize=(10,8))
sns.heatmap(loadings.iloc[:, :5], cmap="coolwarm",center=0)
plt.title("Feature Contributions to Principal Components")
plt.xlabel("Principal Components")
plt.ylabel("Original Features")
plt.show()
```

Output:

Feature Contributions to Principal Components

**# Interactively analyze and visualize top feature contributions for a selected principal component**


```
def interactive_pca(pc_num=1):
    pc = f"PC{pc_num}"
    print(f"Principal Component: {pc}")
    print(f"Explained Variance Ratio: {pca.explained_variance_ratio_[pc_num-1]:.4f}")
    print(f"Cumulative Variance till {pc}: {pca.explained_variance_ratio_[:pc_num].sum():.4f}")
    top_features = loadings[pc].abs().sort_values(ascending=False).head(10)
    plt.figure(figsize=(8,4))
    sns.barplot(x=top_features.values, y=top_features.index)
    plt.xlim(0, top_features.max() * 1.1)
    plt.xlabel("Feature Contribution Strength")
    plt.ylabel("Features")
    plt.title(f"Top 10 Feature Contributions to {pc}")
    plt.grid(True)
    plt.show()

widgets.interact(interactive_pca,
    pc_num=widgets.IntSlider(min=1,max=10,step=1,value=1,description="PCA Component"));
```

Output:

Select any PCA component to visualize its feature contributions:

PCA Comp...

1

Principal Component: PC1

Explained Variance Ratio: 0.4286

Cumulative Variance till PC1: 0.4286

